

УНИВЕРСИТЕТ ИТМО

Факультет прикладной информатики
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа «Мобильные и сетевые технологии»
Дисциплина «Проектная деятельность»



ОТЧЕТ по
Практической работе 9
ML-система автоматического анализа тональности комментариев в социальной
сети
«SentimentGuard AI»

Состав команды:

- Дементьев Антон Павлович — Product Owner,
- Фоченков Семен Андреевич — Data Engineer, Software Architect
- Бочарников Максим Андреевич — Data Scientist, Data Architect

Группа: K3341

Поток: ПИС 2.3

Ссылка на репозиторий:

Санкт-Петербург, 2026 г.

Общая информация

Кейс: Кейс №1 — Автоматизация мониторинга пользовательских реакций на контент в социальной сети

Название работы: Дизайн-документ ML-системы автоматического анализа тональности комментариев в социальной сети (SentimentGuard AI).

Цель работы: Спроектировать ML-систему для автоматической классификации тональности пользовательских комментариев в социальной сети — от постановки задачи и методологии до архитектуры production-системы и критериев успеха пилота.

Задачи работы:

- Сформулировать цели, бизнес-требования и ограничения для системы анализа тональности.
- Описать методологию: тип задачи ML, необходимые данные, метрики качества и риски.
- Спланировать процесс пилотного запуска с критериями успеха.
- Спроектировать архитектуру производственной системы с описанием компонентов, инфраструктуры, SLA и рисков.
- Создать полный комплект архитектурных диаграмм: BPMN (AS-IS и TO-BE), ER, системная архитектура, UML компонентов, UML последовательности.

1. Цели и предпосылки

1.1. Зачем идём в разработку продукта?

Социальная сеть ежедневно принимает до 500 000 пользовательских комментариев. Действующий процесс модерации основан на ручном выборочном просмотре: модераторы охватывают не более 3–5% входящего потока, что порождает три критические проблемы.

Регуляторный риск. Роскомнадзор ведёт постоянный мониторинг качества модерации. Каждый пропущенный запрещённый комментарий грозит штрафом по ст. 13.41 КоАП РФ (от 800 тыс. до 4 млн руб.) и, в пределах, блокировкой ресурса.

Экономическая неэффективность. Команда модераторов тратит рабочее время на механический просмотр, вместо того чтобы сосредоточиться на действительно сложных пограничных случаях.

Медленная реакция. Вирусный негативный или запрещённый контент за 10–30 минут распространяется по платформе — ручная модерация физически не успевает.

1.2. Почему станет лучше от использования ML?

Система машинного обучения позволит перейти от выборочного к тотальному покрытию входящего потока. Конкретные эффекты:

- Автоматическая обработка 100% комментариев в режиме реального времени с задержкой менее 2 секунд.
- Немедленная блокировка очевидных нарушений ($\text{confidence} \geq 0.95$) без участия человека.
- Формирование приоритизированной очереди: модераторы работают только с пограничными случаями, снижение нагрузки — на 70–80%.
- Ведение полного аудит-лога с обоснованием каждого решения — доказательная база при проверках РКН.
- Выявление трендов: рост негатива вокруг конкретного контента, координированные атаки.

1.3. Бизнес-требования и ограничения

Бизнес-требования:

- БТ-1. Система должна обрабатывать не менее 500 000 комментариев в сутки в режиме реального времени.
- БТ-2. Доля пропущенных нарушений (False Negative) не должна превышать 5%.
- БТ-3. Все решения фиксируются в неизменяемом аудит-логе для предъявления регулятору.
- БТ-4. Решение разворачивается на инфраструктуре, соответствующей 152-ФЗ (локализация данных на территории РФ).

- БТ-5. Интеграция с существующей платформой без изменения клиентского интерфейса.

Ограничения:

- Временное: MVP готов к пилоту в течение 3 месяцев от старта разработки.
- Инфраструктурное: предпочтительное развёртывание на российских облачных провайдерах (Яндекс.Облако, SberCloud) или on-premise.
- Правовое: система не может блокировать контент без возможности апелляции — предусмотреть интерфейс обжалования.
- Качественное: допустимый уровень ложных блокировок (False Positive) — не более 3% от легитимных комментариев.

1.4. Функциональные и нефункциональные требования

Функциональные требования:

ID	Требование	Описание
ФТ-1	Классификация тональности	Определение класса: позитив / нейтрал / негатив / токсично / запрещённый контент
ФТ-2	Real-time обработка	Классификация в течение 2 сек. с момента публикации комментария
ФТ-3	Авто-блокировка	Скрытие комментариев с confidence ≥ 0.95 по классам «токсично» и «запрещённый контент»
ФТ-4	Очередь на ревью	Приоритизированная очередь для модераторов при $0.5 \leq \text{confidence} < 0.95$
ФТ-5	Дашборд модератора	Интерфейс просмотра, подтверждения/отмены решений с обоснованием (топ-токены SHAP)
ФТ-6	Аналитический дашборд	Агрегированные метрики: тренды тональности, топ-источники негатива, динамика по времени
ФТ-7	Аудит-лог	Неизменяемый журнал всех решений: автоматических и ручных (для РКН)
ФТ-8	Active Learning	Интерфейс разметки пограничных случаев для дообучения модели

Нефункциональные требования:

Требование	Приоритет	Целевое значение
------------	-----------	------------------

Производительность	Критический	Throughput ≥ 10 RPS средний, ≥ 60 RPS пиковый; latency p95 < 2 сек E2E
Доступность	Высокий	SLA 99.9% uptime; RTO < 30 мин; RPO < 5 мин
Безопасность	Высокий	TLS 1.3, шифрование данных at rest, RBAC, соответствие 152-ФЗ
Масштабируемость	Средний	Горизонтальное масштабирование инференс-сервиса до $\times 10$ от базовой нагрузки
Наблюдаемость	Средний	Метрики модели в Prometheus/Grafana; алерты при деградации метрик качества
Объяснимость	Средний	Для каждого решения — топ-3 токена, повлиявших на класс (SHAP)

1.5. Как построить процесс пилота? Критерии успеха

Пилот проводится в течение 4 недель на 20% входящего трафика по схеме Shadow Mode с постепенным переходом в Soft Launch.

Этапы пилота:

1. Недели 1–2: Shadow Mode. ML классифицирует без вмешательства параллельно с модераторами. Решения только логируются — пользователи их не видят. Ежедневный автоматический отчёт расхождений с анализом топ-50 спорных случаев.
2. Неделя 3: Calibration. Настройка порогов confidence на основе данных первых двух недель. Доразметка пограничных случаев старшими модераторами (формирование golden set).
3. Неделя 4: Soft Launch. Авто-блокировка активирована только для confidence ≥ 0.98 — наиболее однозначные нарушения. Мониторинг апелляций от пользователей.

Критерии успешного пилота:

- F1-score по классу «запрещённый контент» ≥ 0.87 .
- Recall по классу «запрещённый контент» ≥ 0.92 — минимизация пропусков для РКН.
- False Positive Rate $\leq 3\%$ — допустимый уровень ложных блокировок.
- Latency p95 (E2E) < 2 сек при нагрузке 20% трафика.
- Снижение ежедневного объёма очереди ручного просмотра $\geq 50\%$.

1.6. Что входит в MVP, а что остаётся техническим долгом?

Входит в MVP	Технический долг (следующие итерации)
Бинарная классификация: допустимо / нарушение (Fast Path для РКН)	Многоклассовая тональность с нюансами (5+ классов)
Очередь на ручную проверку для пограничных случаев	Автоматическое тематическое кластеризование жалоб
Базовый дашборд модератора (список + решение)	Персонализация: индивидуальные профили риска пользователей
Аудит-лог всех решений (авто + ручных)	Детальная аналитика и отчётность для бизнеса
Интеграция через Kafka с существующей платформой	Мультиязычная поддержка (MVP: только русский язык)
Модель: ruBERT-base, fine-tuned на данных платформы	Active Learning pipeline с полуавтоматической разметкой
Базовый мониторинг деградации модели (PSI)	A/B тестирование версий модели в production

Микрорефлексия. Формулировка требований через призму регуляторных рисков (РКН) принципиально изменила приоритеты: Recall по классу нарушений оказывается важнее Precision. Лучше поставить лишний комментарий в очередь модератору, чем пропустить запрещённый контент и получить штраф. Это неочевидный, но логичный вывод из бизнес-контекста — регуляторные ограничения диктуют архитектурные решения сильнее, чем технические соображения.

2. Методология

2.1. Что делаем с технической точки зрения?

С технической точки зрения решаем задачу многоклассовой классификации текстов (Text Multi-Class Classification) с элементами детекции токсичности и hate-speech.

Архитектура решения:

- Основная задача: классификация комментария в один из 5 классов — позитив, нейтрал, негатив, токсично, запрещённый контент.
- Вспомогательная задача (Binary Fast Path): нарушение / не нарушение — для быстрого ответа по жёстким критериям РКН.
- Модельная архитектура MVP: fine-tuning предобученной языковой модели ruBERT-base (или ruRoBERTa) на размеченных данных платформы.
- Rule Engine как дополнение: детерминированный фильтр по словарям запрещённых слов/фраз работает до ML как Fast Path — перехватывает очевидные случаи с latency < 10 мс.
- Перспектива: multi-task learning с общим энкодером для тональности и токсичности одновременно.

2.2. Какие данные нужны? Без чего нельзя решить задачу?

Источник данных	Объём	Важность / Примечание
Исторические комментарии платформы с решениями модераторов	$\geq 500\text{K}$ размеченных записей	КРИТИЧНО. Основной обучающий датасет. Без него задачу не решить.
Открытый датасет RuToxic (GitHub)	~14K примеров	Базовая разметка токсичности на русском языке
RuSentiment Dataset	~30K примеров	Тональность русскоязычных текстов социальных сетей
Список запрещённых слов/фраз РКН	Регулярно обновляется	Hard-rules для очевидных нарушений — дополнение к ML
Данные апелляций пользователей	Накапливается в production	Ценный источник для коррекции ошибок модели (active learning)

Требования к разметке исторических данных: каждый комментарий должен иметь метку от ≥ 2 модераторов; при расхождении — арбитраж третьим. Минимальный объём для fine-tuning ruBERT — 50K примеров с контролируемым качеством (Cohen's Kappa ≥ 0.7 между разметчиками).

2.3. Метрики качества ML и их связь с бизнес-результатом

ML-метрика	Целевое значение	Связь с бизнес-результатом
Recall (запрещённый контент)	≥ 0.92	Минимизация штрафов РКН: каждый пропуск — потенциальный штраф от 800 тыс. руб.
Precision (запрещённый контент)	≥ 0.80	Уменьшение ложных блокировок → удержание пользователей и их доверия
F1-macro (все классы)	≥ 0.82	Общее качество классификатора по всем категориям тональности
ROC-AUC (binary)	≥ 0.94	Качество разделения нарушений от нормального контента
Latency p95 инференса	< 500 мс (ML-сервис)	Составляющая общего SLA системы: суммарная задержка < 2 сек
Throughput	≥ 100 RPS на 1 реплику	Возможность горизонтального масштабирования без деградации латентности
Data Drift (PSI)	< 0.2 еженедельно	Алерт на деградацию модели при изменении языка и сленга платформы

2.4. Риски на этапе анализа и моделирования

Риск	Уровень	Митигация
Дисбаланс классов: нарушения составляют ~1–2% от потока	Высокий	SMOTE / oversampling для редких классов; class_weight в функции потерь; отдельный threshold для каждого класса
Концептуальный дрейф: язык и сленг меняются быстро	Высокий	Еженедельный мониторинг PSI через Evidently AI; автоматический алерт при $PSI > 0.2$; ежемесячное дообучение
Adversarial-атаки: намеренные опечатки, l33t speak	Средний	Аугментация обучающих данных искусственными ошибками; subword-токенизация BPE в ruBERT устойчива к ним
Субъективность разметки (граница токсичного / нейтрального)	Средний	Контроль Inter-annotator agreement (Cohen's Kappa ≥ 0.7); чёткий rubric для разметчиков

Недостаток данных для редких классов нарушений	Средний	Data augmentation (EDA, back-translation); синтетическая генерация через LLM с последующей верификацией
Регуляторная неопределённость: критерии РКН меняются без предупреждения	Низкий	Модульный Rule Engine: словари обновляются независимо от ML-модели через feature flag

Микрорефлексия. Анализ рисков показал, что дисбаланс классов — не техническая мелочь, а системный вызов: при 1% реальной доле нарушений модель может давать 99% ассурасу, не блокируя ничего. Поэтому ключевая метрика — Recall, а не Accuracy. Именно регуляторный контекст РКН делает это решение очевидным: штраф за каждый пропуск делает False Negative несравнимо дороже False Positive.

3. Подготовка пилота

3.1. Способ оценки пилота

Пилот проводится по схеме Shadow Mode с последовательным переходом в Soft Launch. Оценка ведётся по двум осям: техническое качество модели и измеримый бизнес-эффект.

Shadow Mode (Недели 1–2):

- Все входящие комментарии параллельно обрабатываются системой ML, но решения не применяются к пользователям — только логируются.
- Модераторы работают в штатном режиме, не зная о решениях ML (blind evaluation).
- Ежедневно формируется отчёт расхождений; команда ML разбирает топ-50 спорных случаев и корректирует пороги или разметку.

Soft Launch (Неделя 4):

- Авто-блокировка активирована только для confidence ≥ 0.98 — ожидаем ~30% потока нарушений.
- Для остальных нарушений — приоритизированная очередь модераторам с сортировкой по confidence.
- Пользователь получает уведомление о блокировке с возможностью подать апелляцию.

3.2. Что считаем успешным пилотом

Критерий	Целевой порог	Метод измерения
F1-score (запрещённый контент)	≥ 0.87	Offline: на тестовой выборке размеченных данных пилота
Recall (запрещённый контент)	≥ 0.92	Сравнение с решениями старших модераторов (golden set)
False Positive Rate	$\leq 3\%$	Доля жалоб пользователей, признанных ошибочными при апелляции
Latency p95 E2E	< 2 сек	Мониторинг в Prometheus: от публикации до решения системы
Снижение очереди ручного просмотра	$\geq 50\%$	Сравнение среднесуточного объёма очереди модераторов до и после
Согласие с решением модератора	$\geq 80\%$	Random sample 1000 комментариев в день, проверенных вручную

3.3. Подготовка пилота

Состав пилотной команды:

- ML-инженер (×1): мониторинг метрик, быстрые итерации модели по результатам каждого дня.
- Старший модератор (×2): разметка спорных случаев, формирование и поддержка golden set.
- DevOps-инженер (×1): мониторинг инфраструктуры, масштабирование при пиковых нагрузках.
- Product Owner: еженедельные стендапы, приоритизация доработок, коммуникация с бизнесом.

Инфраструктура пилота:

- Среда: Kubernetes-кластер на Яндекс.Облако (Managed Kubernetes), 20% реального трафика.
- Модель: ruBERT-base, fine-tuned на 100K исторических размеченных комментариев платформы.
- Мониторинг: Prometheus + Grafana с ключевыми метриками; еженедельный Quality Report.
- Откат: при деградации ниже порогов — автоматический переход в режим «только очередь» без авто-блокировок (feature flag в Decision Orchestrator).

Микрорефлексия. Схема Shadow Mode выбрана осознанно: она позволяет собрать реальную оценочную выборку без риска негативного воздействия на пользователей. Главный инсайт планирования пилота — golden set должен формироваться операционными модераторами, а не разработчиками ML: иначе модель оценивается на данных, которые сами же и были использованы для настройки.

4. Внедрение для production-системы

4.1. Архитектура решения

Система SentimentGuard AI построена на событийно-ориентированной микросервисной архитектуре. Центральный транспорт — Apache Kafka, обеспечивающий декуплинг компонентов и устойчивость к сбоям. ML-инференс вынесен в отдельный сервис для независимого масштабирования.

Компоненты системы и их взаимосвязи:

Компонент	Технология	Ответственность
Comment Ingestion Service	FastAPI + Kafka Producer	Приём новых комментариев с платформы, публикация событий в топик comments.raw
Rule Engine	Python (Aho-Corasick)	Hard-rules по словарям РКН. Fast Path: блокировка очевидных нарушений за < 10 мс до ML
ML Inference Service	FastAPI + PyTorch + ruBERT	Классификация текста: 5-классовая тональность + confidence score. Основной ML-компонент.
Decision Orchestrator	FastAPI + Kafka Consumer	Агрегирует результаты Rule Engine и ML, принимает финальное решение по порогу confidence
Moderation Queue Service	PostgreSQL + Redis	Управление очередью ручной проверки, приоритизация по confidence, WebSocket-уведомления
Moderator Dashboard	React SPA + REST API	UI для модераторов: просмотр очереди, вынесение решений, апелляции, SHAP-объяснения
Analytics Dashboard	ClickHouse + Grafana	Бизнес-аналитика: тренды тональности, эффективность модерации, отчётность для РКН
Audit Service	ClickHouse (append-only)	Неизменяемый аудит-лог всех решений — автоматических и ручных
Model Monitoring Service	Prometheus + Evidently AI	Контроль дрейфа данных (PSI), деградации метрик, алерты при нарушении порогов

ML Training Pipeline	Airflow + MLflow	Переобучение модели по расписанию или по триггеру дрейфа; версионирование моделей
----------------------	------------------	---

Основной поток обработки (Happy Path):

4. Пользователь публикует комментарий → платформа отправляет событие в Comment Ingestion Service.
5. Ingestion Service публикует сообщение в топик Kafka: comments.raw.
6. Rule Engine (Fast Path): проверяет по словарям запрещённых слов. При совпадении — немедленная блокировка, минуя ML (< 10 мс).
7. ML Inference Service потребляет из Kafka, классифицирует текст, возвращает вектор вероятностей по 5 классам.
8. Decision Orchestrator: $\text{confidence} \geq 0.95 \rightarrow$ авто-блок; $0.5 \leq \text{confidence} < 0.95 \rightarrow$ очередь; $< 0.5 \rightarrow$ пропуск.
9. Решение записывается в PostgreSQL (оперативные данные) и ClickHouse (аудит-лог).
10. Если решение «очередь» → событие в Kafka: moderation.queue → WebSocket-уведомление модератору.
11. Модератор принимает финальное решение через Dashboard → аудит-лог обновляется.

4.2. Описание инфраструктуры и масштабируемость

Развёртывание на Яндекс.Облако (Managed Kubernetes + Managed PostgreSQL + Managed Kafka + Object Storage). Это соответствует требованиям 152-ФЗ о локализации персональных данных граждан РФ.

Компонент инфраструктуры	Плюсы	Минусы / Риски
Managed Kubernetes (YC)	Автомасштабирование, управляемые обновления, интеграция с YC-сервисами	Vendor lock-in, стоимость при высокой нагрузке
Managed Kafka (YC)	Не нужно администрировать кластер самостоятельно	Ограниченная конфигурируемость vs self-hosted
GPU-ноды для ML Inference	Ускорение BERT-инференса в $10\text{--}20\times$ vs CPU	Высокая стоимость; необходима оптимизация: batching, ONNX-квантизация

ClickHouse (self-hosted в K8s)	Полный контроль, оптимальная производительность аналитики и аудита	Требует DevOps-компетенций для эксплуатации
Redis (Managed YC)	Кэш последних решений и сессий, latency < 1 мс	Данные персистируются только при AOF/RDB — возможна потеря при сбое

Стратегия масштабирования:

- ML Inference Service: HPA по метрике CPU и custom metric (Kafka consumer lag). При lag > 1000 сообщений — автодобавление реплик. Максимум: 10 реплик.
- Batching: группировка запросов в батчи по 32 текста с max wait timeout 50 мс — рост throughput с 10 до 100+ RPS на реплику.
- Model Quantization (INT8): снижение размера модели с 420 MB до ~120 MB, ускорение CPU-инференса в 2–3×.
- Rule Engine: stateless, масштабируется горизонтально без ограничений — отдельный Deployment.

4.3. Требования к работе системы: SLA, RPS, Latency

Параметр	Целевое значение	Примечание
Availability (SLA)	99.9%	Менее 8.7 часов суммарного простоя в год
RTO (Recovery Time Objective)	< 30 минут	Время восстановления после инцидента
RPO (Recovery Point Objective)	< 5 минут	Максимальная потеря данных (Kafka retention 7 дней)
Throughput (средний)	≥ 10 RPS	500К комментариев / 86400 сек ≈ 5.8 RPS; запас ×2
Throughput (пиковый)	≥ 60 RPS	Пиковые часы: ~10× от среднего; HPA обеспечивает горизонтальный рост
E2E Latency p95	< 2 сек	От момента публикации комментария до финального решения системы
ML Inference latency p95	< 500 мс	Инференс одного текста (batch=1). GPU-нода: < 50 мс

4.4. Риски: неопределённости, которые стоит предусмотреть

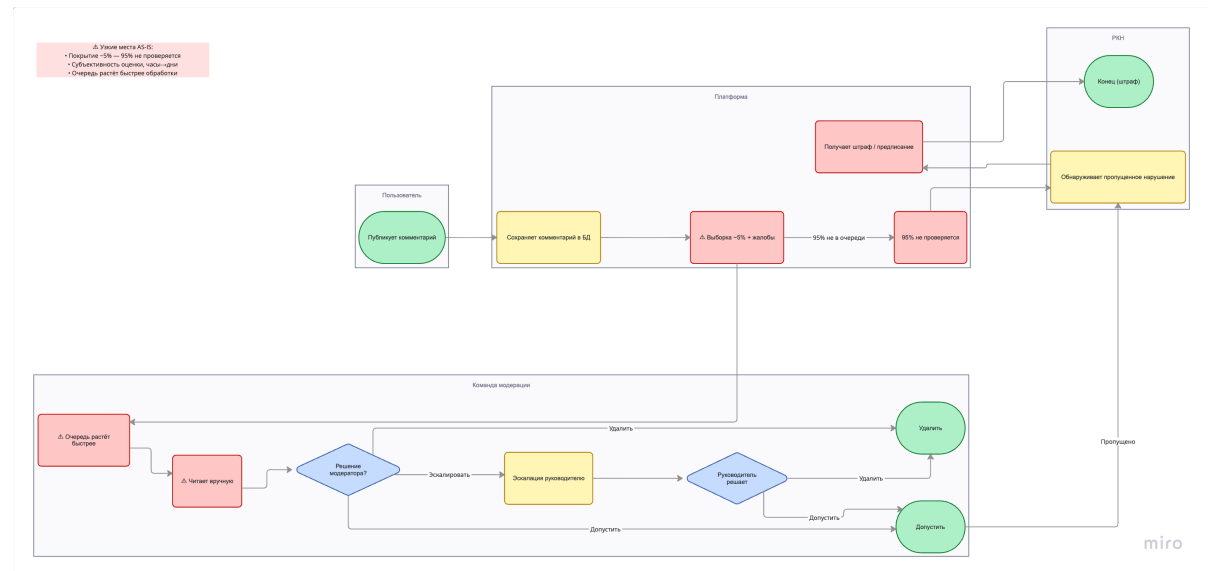
Риск	Уровень	Планируемые меры
Деградация качества модели при дрейфе данных (новый сленг, события)	Высокий	Еженедельный мониторинг PSI в Evidently AI; алерт при $PSI > 0.2 \rightarrow$ переобучение по триггеру
Отказ GPU-ноды в период пиковой нагрузки	Средний	HPA + CPU fallback: при недоступности GPU — автопереключение на CPU-реплики с квантизованной моделью
Рост Kafka lag при вирусном посте (spike-нагрузка)	Средний	Retention 7 дней — данные не теряются; масштабирование консьюмеров по кастомной метрике lag
Изменение критериев РКН без предупреждения	Средний	Модульный Rule Engine: обновление словарей через feature flag без деплоя ML-модели
Утечка персональных данных пользователей	Высокий	Данные не покидают YC; шифрование at rest и in transit; RBAC на все хранилища; аудит доступа
Оспаривание автоматических блокировок регулятором или пользователем	Средний	Каждое решение логируется с probability scores + топ-токены SHAP; апелляционный интерфейс

Микрорефлексия. Наибольшая неопределённость в production связана не с технической, а с регуляторной стороной: критерии РКН не имеют однозначной формальной спецификации. Архитектурное разделение Rule Engine (обновляемые правила) и ML-модели (статистические паттерны) позволяет реагировать на регуляторные изменения оперативно, не запуская переобучение модели. Это ключевое архитектурное решение, продиктованное не технической необходимостью, а бизнес-контекстом.

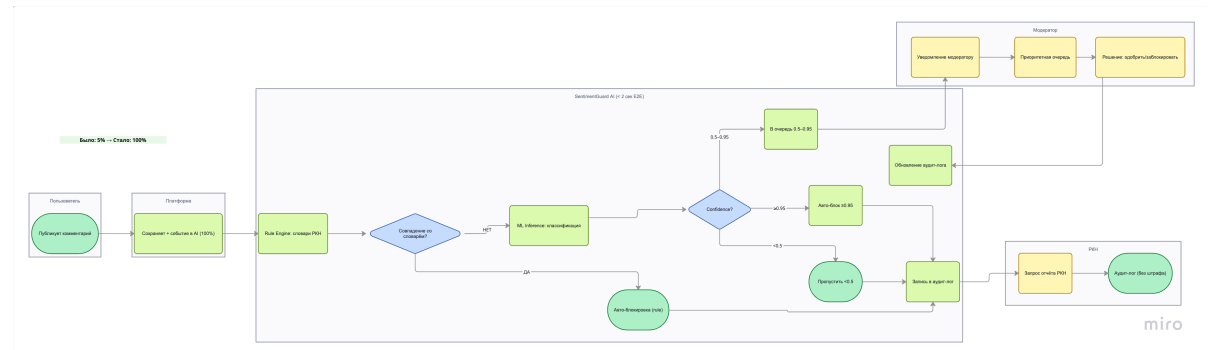
5. Диаграммы

5.1. BPMN — Бизнес-процессы (AS-IS и TO-BE)

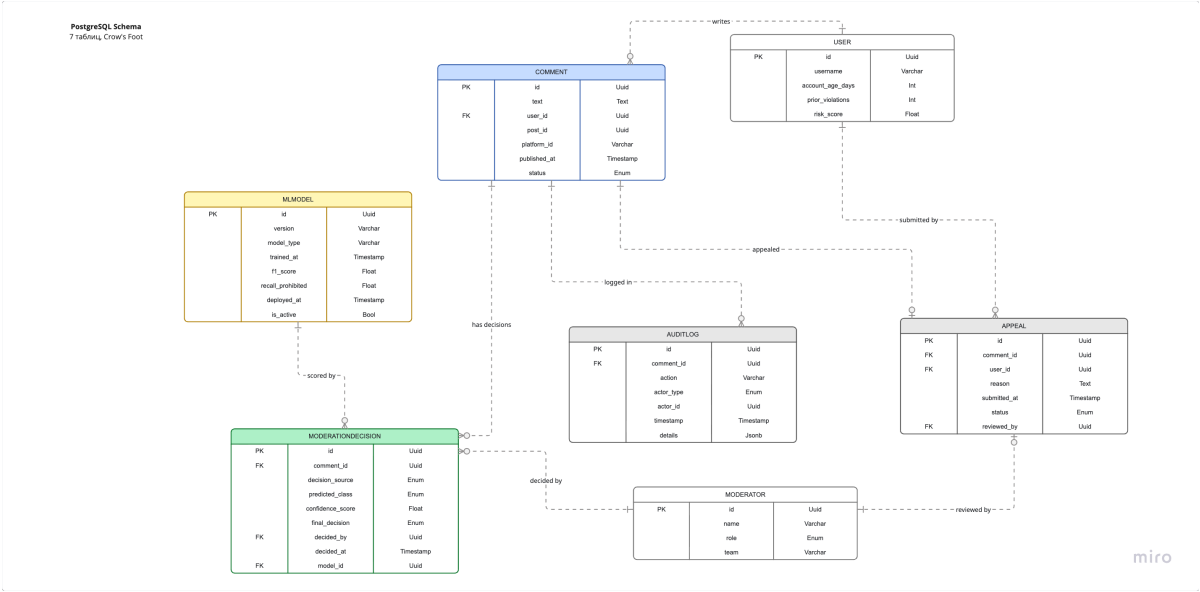
BPMN AS-IS



BPMN TO-BE



5.2. ER-диаграмма структуры данных



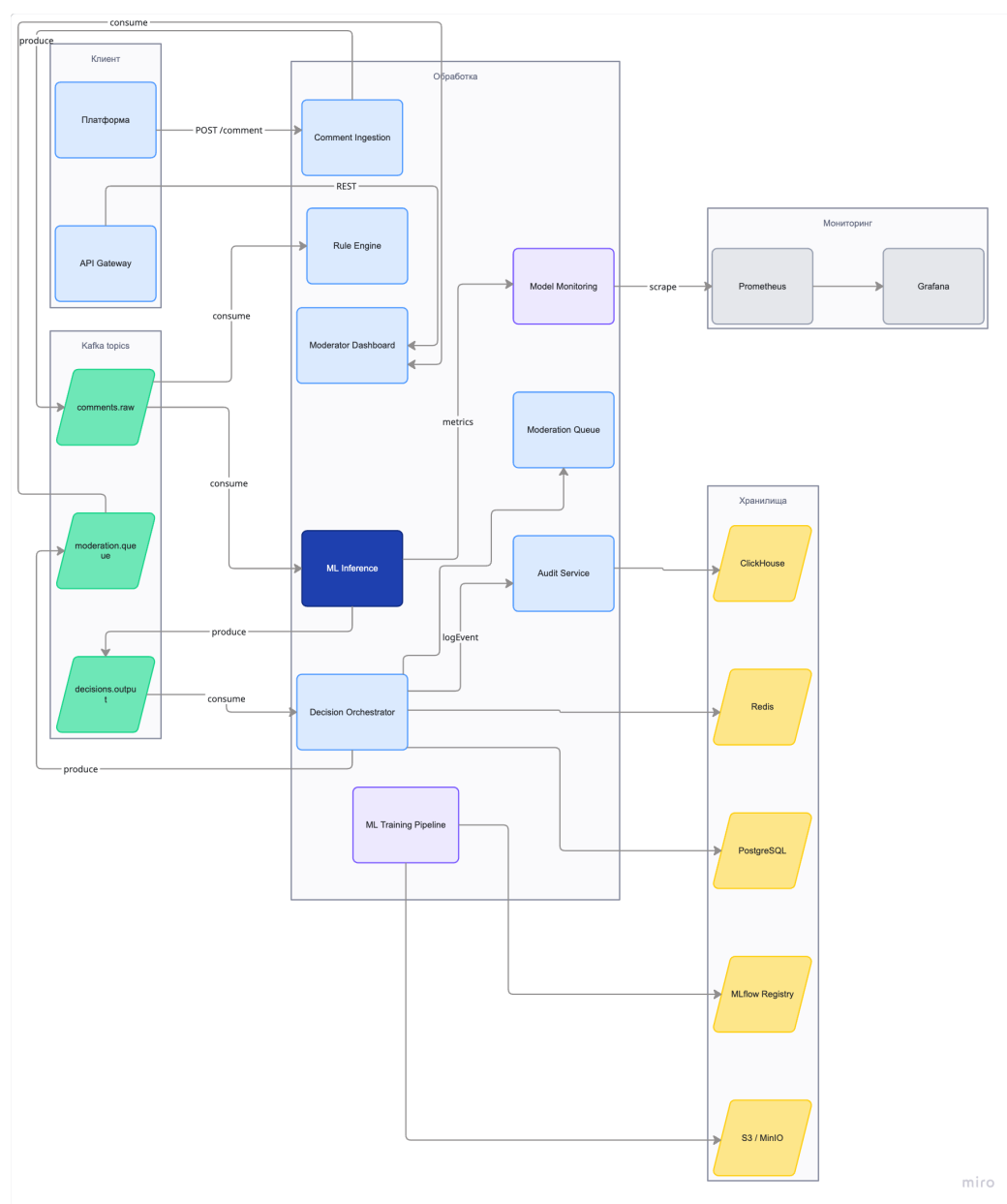
Сущности и их атрибуты:

Сущность	Ключевые атрибуты
Comment	id (UUID, PK), text (TEXT), user_id (FK), post_id (UUID), published_at (TIMESTAMP), status (ENUM: active/blocked/pending)
ModerationDecision	id (UUID, PK), comment_id (FK), decision_source (ENUM: ml/rule/human), predicted_class (ENUM: 5 классов), confidence_score (FLOAT), final_decision (ENUM), decided_by (FK, nullable), decided_at (TIMESTAMP)
User	id (UUID, PK), username, account_age_days, prior_violations_count, risk_score (FLOAT)
Moderator	id (UUID, PK), name, role (ENUM: moderator/senior/admin), team
MLModel	id (UUID, PK), version (VARCHAR), model_type, trained_at, fl_score, recall_prohibited, is_active (BOOL)
AuditLog	id (UUID, PK), comment_id (FK), action (VARCHAR), actor_type (ENUM), actor_id, timestamp, details (JSONB)
Appeal	id (UUID, PK), comment_id (FK), user_id (FK), reason (TEXT), submitted_at, status (ENUM), reviewed_by (FK)

Связи между сущностями:

- Comment (1) → ModerationDecision (N) — один комментарий может иметь несколько решений (авто + ручной пересмотр).
- User (1) → Comment (N) — один пользователь пишет много комментариев.
- Moderator (1) → ModerationDecision (N) — один модератор принимает много решений.
- Comment (1) → AuditLog (N) — все действия с комментарием логируются.
- Comment (1) → Appeal (0..1) — пользователь может подать апелляцию на блокировку.
- MLModel (1) → ModerationDecision (N) — фиксируется версия модели, принявшей решение.

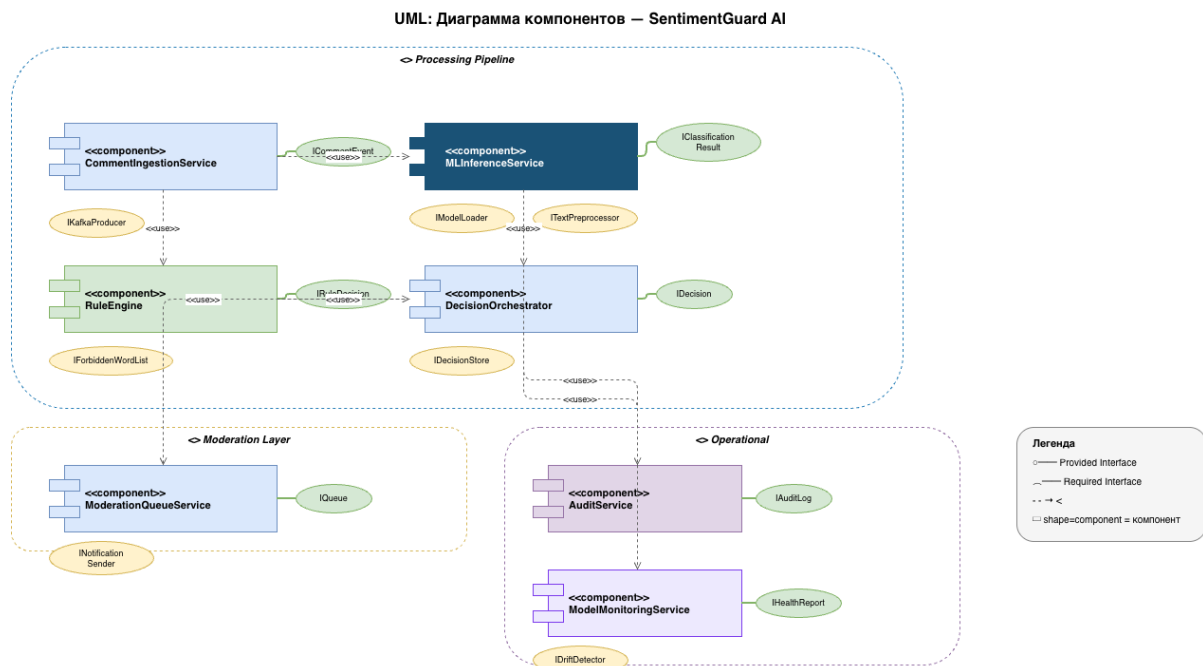
5.3. Архитектурная диаграмма системы



Группировки (зоны) на диаграмме:

- Зона 1 «Клиентский уровень»: иконки браузера и мобильного приложения → API Gateway.
- Зона 2 «Сервисный уровень» (Kubernetes): Comment Ingestion, Rule Engine, ML Inference, Decision Orchestrator, Moderation Queue, Moderator Dashboard, Audit Service, Model Monitoring.
- Зона 3 «Транспорт»: Apache Kafka с топиками comments.raw, decisions.output, moderation.queue.
- Зона 4 «Хранилища»: PostgreSQL, Redis, ClickHouse, S3/MinIO, MLflow Registry.
- Зона 5 «MLOps»: Airflow (Training Pipeline), MLflow.
- Зона 6 «Внешние системы»: Платформа (источник комментариев), инфраструктура Яндекс.Облако.

5.4. UML-диаграмма компонентов



Компоненты и интерфейсы:

- <<component>> CommentIngestionService — provides: ICommentEvent; requires: IKafkaProducer
- <<component>> RuleEngine — provides: IRuleDecision; requires: IForbiddenWordList
- <<component>> MLInferenceService — provides: IClassificationResult; requires: IModelLoader, ITextPreprocessor
- <<component>> DecisionOrchestrator — provides: IDecision; requires: IRuleDecision, IClassificationResult, IDecisionStore
- <<component>> ModerationQueueService — provides: IQueue; requires: IDecision, INotificationSender
- <<component>> AuditService — provides: IAuditLog; requires: IDecision
- <<component>> ModelMonitoringService — provides: IHealthReport; requires: IClassificationResult, IDriftDetector

5.5. UML-диаграмма последовательности

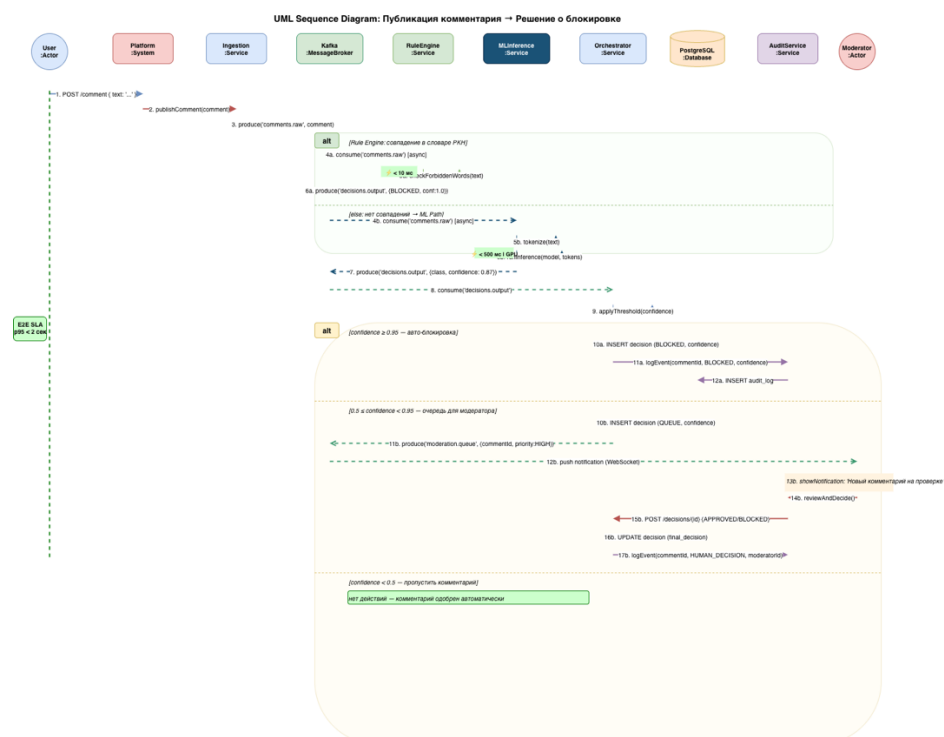
Сценарий: «Публикация комментария и принятие решения о блокировке».

Участники (Lifelines):

User : Actor | Platform : System | CommentIngestion : Service | Kafka : MessageBroker
 | RuleEngine : Service | MLInference : Service | DecisionOrchestrator : Service |
 PostgreSQL : Database | AuditService : Service | Moderator : Actor

Последовательность сообщений:

1. User → Platform: POST /comment { text: '...' }
2. Platform → CommentIngestion: publishComment(comment)
3. CommentIngestion → Kafka: produce(comments.raw, comment)
4. Kafka → RuleEngine: consume(comment) [async]
5. alt [совпадение в словаре]:
6. RuleEngine → Kafka: produce(decisions.output, {BLOCKED, confidence:1.0})
7. else [нет совпадений]:
8. Kafka → MLInference: consume(comment) [async]
9. MLInference → MLInference: tokenize(text) + inference(model)
10. MLInference → Kafka: produce(decisions.output, {class, confidence:0.xx})
11. Kafka → DecisionOrchestrator: consume(decisions.output)
12. alt [confidence ≥ 0.95]:
13. DecisionOrchestrator → PostgreSQL: save(BLOCKED)
14. DecisionOrchestrator → AuditService: logEvent(BLOCKED, confidence)
15. alt [0.5 ≤ confidence < 0.95]:
16. DecisionOrchestrator → PostgreSQL: save(Queue)
17. DecisionOrchestrator → Kafka: produce(moderation.queue, {commentId, priority})
18. Kafka → ModeratorDashboard: push notification
19. Moderator → ModeratorDashboard: reviewComment() → decision
20. ModeratorDashboard → DecisionOrchestrator: submitDecision(commentId, APPROVED/BLOCKED)
21. DecisionOrchestrator → AuditService: logEvent(HUMAN_DECISION)



Микрорефлексия. Построение sequence-диаграммы оказывается наиболее трудоёмким, но и наиболее ценным упражнением: именно в ходе её создания обнаруживаются неочевидные граничные случаи — что происходит, если ML-сервис не отвечает, если Kafka недоступна, если Decision Orchestrator падает между записью в PostgreSQL и публикацией события? Диаграмма фактически является спецификацией для разработчиков каждого из сервисов.

6. Структура репозитория

Содержимое
Мастер-документ (данный дизайн-документ в Markdown)
BPMN: текущий процесс модерации (AS-IS)
BPMN: процесс с ML-системой (TO-BE)
Текстовое описание обеих BPMN-диаграмм
ER-диаграмма структуры данных
Описание ER: нормализация, индексы, обоснование
Архитектурная диаграмма системы
Описание архитектуры, обоснование решений
UML-диаграмма компонентов
UML-диаграмма последовательности
Описание обеих UML-диаграмм
Детальный план пилота: расписание, метрики, команда

7. Заключение

В ходе работы разработан полный дизайн-документ ML-системы SentimentGuard AI для автоматического анализа тональности комментариев в социальной сети, охватывающий все этапы от постановки задачи до производственной архитектуры.

Результаты:

- Сформулированы бизнес-требования и ограничения с учётом регуляторного контекста РКН; определён MVP и технический долг.
- Описана методология: fine-tuning ruBERT для 5-классовой классификации тональности; определены ML-метрики с явной привязкой к бизнес-результату.
- Спроектирован пилот по схеме Shadow Mode → Calibration → Soft Launch с измеримыми критериями успеха.
- Разработана событийно-ориентированная архитектура из 10 компонентов на Kafka; описано распределённое хранилище с обоснованием выбора каждого хранилища.

Ключевой вывод: регуляторные ограничения (РКН) диктуют архитектурные решения сильнее, чем технические соображения. Разделение Rule Engine и ML-модели, приоритет Recall над Precision, обязательный аудит-лог — всё это следствия не технической необходимости, а бизнес-контекста.